

Writing excellent requirements

"Hi, Gautam. This is Ruth calling from the Austin branch. We got that latest drop of the website software for the online music store. I wanted to ask you about the song preview feature. That's not working the way I had in mind."

Gautam replied, "Let me find the requirements you sent for that. Here they are. The user story said, 'As a Customer, I want to listen to previews of the available songs so I can decide which ones to buy.' My notes say that when we discussed this, you said each song sample should be 30 seconds long and that it should use our built-in MP3 player so the customer didn't have to wait for another player to launch. Isn't that working correctly?"

"Well, yes, that all works fine," said Ruth, "but there are a couple of problems. I can click on the play icon to start the sample, but I don't have any way to pause it or stop it. I'm forced to listen to the entire 30-second sample. Also, all the samples start at the beginning of the song. Some songs have long introductions so you really can't hear what they're like from just the beginning. The sample should start somewhere in the middle of those songs so people could hear what they're really like. And the sample starts playing at full volume and then stops abruptly. If the customer's speakers are up pretty loud this could be startling. I think it would be better if we fade in and fade out on each sample."

Gautam was a little frustrated. "I wish you had told me all of this when we spoke earlier. You didn't give me much to go on so I just had to make my best guess. I can do all that, but it's going to take a few more days."

The best requirements repository in the world is useless if it doesn't contain high-quality information. This chapter describes desirable characteristics of requirements and of requirements documents. It presents numerous guidelines for writing requirements, along with many examples of flawed requirements and suggestions for improving them. These recommendations apply to the requirements that are created for any project following any development life cycle. The requirements authors on each project need to judge the appropriate level of precision and detail for their requirements, but there's no substitute for clear communication.

Characteristics of excellent requirements

How can you tell good requirements from those with problems? This section describes several characteristics that individual requirement statements should exhibit, followed by desirable characteristics of the requirements set as a whole (Davis 2005; ISO/IEC/IEEE 2011). The best way

to tell whether your requirements possess these desired attributes is to have several stakeholders review them. Different stakeholders will spot different kinds of problems. Chapter 17, “Validating the requirements,” describes the use of checklists to remind reviewers of common requirements errors.

Characteristics of requirement statements

In an ideal world, every individual business, user, functional, and nonfunctional requirement would exhibit the qualities described in the following sections.

Complete

Each requirement must contain all the information necessary for the reader to understand it. In the case of functional requirements, this means providing the information the developer needs to be able to implement it correctly. If you know you’re lacking certain information, use *TBD* (to be determined) as a standard flag to highlight these gaps, or log them in an issue-tracking system to follow up on later. Resolve all TBDs in each portion of the requirements before the developers proceed with construction of that portion.

Correct

Each requirement must accurately describe a capability that will meet some stakeholder’s need and must clearly describe the functionality to be built. You’ll have to go to the source of the requirement to check its correctness. This might be a user who supplied the initial requirement, a higher-level system requirement, a use case, a business rule, or another document. A low-level requirement that conflicts with its parent is not correct. To assess the correctness of user requirements, user representatives or their close surrogates should review them.

Feasible

It must be possible to implement each requirement within the known capabilities and limitations of the system and its operating environment, as well as within project constraints of time, budget, and staff. A developer who participates during elicitation can provide a reality check on what can and cannot be done technically and what can be done only at excessive cost or effort. Incremental development approaches and proof-of-concept prototypes are two ways to evaluate requirement feasibility. If a requirement needs to be cut because it is not be feasible, understand the impact on the project vision and scope.

Necessary

Each requirement should describe a capability that provides stakeholders with the anticipated business value, differentiates the product in the marketplace, or is required for conformance to an external standard, policy, or regulation. Every requirement should originate from a source that has the authority to provide requirements. Trace functional and nonfunctional requirements back to specific voice-of-the-user input, such as a use case or user story. You should be able to relate each

requirement to a business objective that clearly indicates why it's necessary. If someone asks why a particular requirement is included, there should be a good answer.

Prioritized

Prioritize business requirements according to which are most important to achieving the desired value. Assign an implementation priority to each functional requirement, user requirement, use case flow, or feature to indicate how essential it is to a particular product release. If all requirements are equally important, the project manager doesn't know how best to respond to schedule overruns, personnel losses, or new requirements that come along. Requirements prioritization should be a collaborative activity involving multiple stakeholder perspectives. Chapter 16, "First things first: Setting requirement priorities," discusses prioritization in further detail.

Unambiguous

Natural language is prone to two types of ambiguity. One type I can spot myself, when I can think of more than one way to interpret a given requirement. The other type of ambiguity is harder to catch. That's when different people read the requirement and come up with different interpretations of it. The requirement makes sense to each of them but means something different to each of them. Inspections are a good way to spot ambiguities (Wiegers 2002). A formal peer review such as an inspection (as opposed to just passing out the requirements to individuals to examine on their own) provides an opportunity for each participant to compare his understanding of each requirement to someone else's. "Comprehensible" is related to "unambiguous": readers must understand what each requirement is saying. Chapter 17 describes the software peer review process.

You'll never remove all the ambiguity from requirements—that's the nature of human language. Most of the time, reasonable people can draw the right conclusions from even a slightly fuzzy requirement. Getting a little help from your colleagues through reviews will clean up a lot of the worst issues, though.

Verifiable

Can a tester devise tests or other verification approaches to determine whether each requirement is properly implemented? If a requirement isn't verifiable, deciding whether it was correctly implemented becomes a matter of opinion, not objective analysis. Requirements that are incomplete, inconsistent, infeasible, or ambiguous are also unverifiable. Testers are good at examining requirements for verifiability. Include them in your requirements peer reviews to catch problems early.

Characteristics of requirements collections

It's not enough to have excellent individual requirement statements. Sets of requirements that are grouped into a baseline for a specific release or iteration should exhibit the characteristics described in the following sections, whether they are recorded in an SRS document, a requirements management tool, a set of user stories and acceptance tests, or any other form.

Complete

No requirement or necessary information should be absent. In practice, you'll never document *every* single requirement for any system. There are always some assumed or implied requirements, although they carry more risk than explicitly stated requirements. Missing requirements are hard to spot because they aren't there! The section "Avoiding incompleteness" later in this chapter suggests some ways to identify missing requirements. Any specification that contains TBDs is incomplete.

Consistent

Consistent requirements don't conflict with other requirements of the same type or with higher-level business, user, or system requirements. If you don't resolve contradictions between requirements before diving into construction, the developers will have to deal with them. Recording the originator of each requirement lets you know who to talk to if you discover conflicts. It can be hard to spot inconsistencies when related information is stored in different locations, such as in a vision and scope document and in a requirements management tool.

Modifiable

You can always rewrite a requirement, but you should maintain a history of changes made to each requirement, especially after they are baselined. You also need to know about connections and dependencies between requirements so you can find all the ones that must be changed together. Modifiability dictates that each requirement be uniquely labeled and expressed separately from others so you can refer to it unambiguously. See Chapter 10, "Documenting the requirements," for various ways to label requirements.

To facilitate modifiability, avoid stating requirements redundantly. Repeating a requirement in multiple places where it logically belongs makes the document easier to read but harder to maintain (Wiegers 2006). The multiple instances of the requirement all have to be modified at the same time to avoid generating inconsistencies. Cross-reference related items in the SRS to help keep them synchronized when making changes. Storing individual requirements just once in a requirements management tool solves the redundancy problem and facilitates reuse of common requirements across multiple projects. Chapter 18, "Requirements reuse," offers several strategies for reusing requirements.

Traceable

A traceable requirement can be linked both backward to its origin and forward to derived requirements, design elements, code that implements it, and tests that verify its implementation. Note that you don't actually have to define all of these trace links for a requirement to have the properties that make it traceable. Traceable requirements are uniquely labeled with persistent identifiers. They are written in a structured, fine-grained way, not in long narrative paragraphs. Avoid combining multiple requirements together into a single statement, because the different requirements might trace to different development components. Chapter 29, "Links in the requirements chain," addresses requirements tracing.

You're never going to create a perfect specification in which *all* requirements demonstrate *all* of these ideal attributes. But if you keep these characteristics in mind when you write and review the requirements, you'll produce better requirements specifications and better software.

Guidelines for writing requirements

There is no formulaic way to write excellent requirements; the best teachers are experience and feedback from the recipients of your requirements. Receiving constructive feedback from colleagues with sharp eyes is a great help because you can learn where your writing did and didn't hit the mark. This is why peer reviews of requirements documents are so critical. To get started with reviews, buddy up with a fellow business analyst and begin exchanging requirements for review. You'll learn from seeing how another BA writes requirements, and you'll improve the team's collective work by discovering errors and improvement opportunities as early as possible. The following sections provide numerous tips for writing requirements—particularly functional requirements—that readers can clearly understand. Benjamin Kovitz (1999), Ian Alexander and Richard Stevens (2002), and Karl Wiegers (2006) present many other recommendations and examples for writing good requirements.

When we say "writing requirements," people immediately think of writing textual requirements in natural language. It's better to mentally translate the phrase "writing requirements" to "representing requirements knowledge." In many cases, alternative representation techniques can present information more effectively than can straight text (Wiegers 2006). The BA should choose an appropriate mix of communication methods that ensures a clear, shared understanding of both the stakeholder needs and the solution to be built.

The sample requirements presented here can always be improved upon, and there are always equivalent ways to state them. Two important goals of writing requirements are that:

- Anyone who reads the requirement comes to the same interpretation as any other reader.
- Each reader's interpretation matches what the author intended to communicate.

These outcomes are more important than purity of style or conforming dogmatically to some arbitrary rule or convention.

System or user perspective

You can write functional requirements from the perspective of either something the system does or something the user can do. Because effective communication is the overarching goal, it's fine to intermingle these styles, phrasing each requirement in whichever style is clearer. State requirements in a consistent fashion, such as "The system shall" or "The user shall," followed by an action verb, followed by the observable result. Specify the trigger action or condition that causes the system to perform the specified behavior. A generic template for a requirement written from the system's perspective is (Mavin et al. 2009):

[optional precondition] [optional trigger event] the system shall [expected system response].

This template is from the Easy Approach to Requirements Syntax (EARS). EARS also includes additional template constructs for event-driven, unwanted behavior, state-driven, optional, and complex requirements. Following is an example of a simple functional requirement that describes a system action using this template:

If the requested chemical is found in the chemical stockroom, the system shall display a list of all containers of the chemical that are currently in the stockroom.

This example includes a precondition, but not a trigger. Some requirement writers would omit the phrase “the system shall” from this requirement. They argue that, because the requirements are describing system behavior, there’s no need to repetitively say “the system shall” do this or that. In this example, deleting “the system shall” is not confusing. Sometimes, though, it’s more natural to phrase the requirement in terms of a user’s action, not from the system’s perspective. Including the “shall” and writing in the active voice makes it clear what entity is taking the action described.

When writing functional requirements from the user’s perspective, the following general structure works well (Alexander and Stevens 2002):

The [user class or actor name] shall be able to [do something] [to some object] [qualifying conditions, response time, or quality statement].

Alternative phrasings are “The system shall let (or allow, permit, or enable) the [a particular user class name] to [do something].” Following is an example of a functional requirement written from the user’s perspective:

The Chemist shall be able to reorder any chemical he has ordered in the past by retrieving and editing the order details.

Notice how this requirement uses the name of the user class—Chemist—in place of the generic “user.” Making the requirement as explicit as possible reduces the possibility of misinterpretation.

Writing style

Writing requirements isn’t like writing either fiction or other types of nonfiction. The writing style you might have learned in school in which you present the main idea, then supporting facts, then the conclusion, doesn’t work well. Adjust your writing style to put the punch line—the statement of need or functionality—first, followed by supporting details (rationale, origin, priority, and other requirement attributes). This structure helps readers who are just skimming through a document, while still being useful for those thorough readers who need all the details. Including tables, structured lists, diagrams, and other visual elements helps to break up a monotonous litany of functional requirements and provides richer communication to those who learn best in different ways.

Nor are requirements documents the place to practice your creative writing skills. Avoid interleaving passive and active voice in an attempt to make the material more interesting to read. Don’t use multiple terms for the same concept just to achieve variety (customer, account, patron, user, client). Being easy to read and understand is an essential element of well-written requirements; being interesting is, frankly, less important. If you are not a skilled writer, you should expect that your

readers might not understand what you intend to convey. Keep the tips that follow in mind as you craft your requirements statements for maximum communication effectiveness.

Clarity and conciseness Write requirements in complete sentences using proper grammar, spelling, and punctuation. Keep sentences and paragraphs short and direct. Write requirements in simple and straightforward language appropriate to the user domain, avoiding jargon. Define specialized terms in a glossary.

Another good guideline is to write concisely. Phrases like “needs to provide the user with the capability to” can be condensed to “shall.” For each piece of information in the requirements set, ask yourself, “What would the reader do with this information?” If you aren’t certain that some stakeholder would find that information valuable, perhaps you don’t need it. Clarity is more important than conciseness, though.

Precisely stated requirements increase the chance of people receiving what they expect; less specific requirements offer the developer more latitude for interpretation. Sometimes that lack of specificity is fine, but in other cases it can lead to too much variability in the outcome. If a developer who reviews the SRS isn’t clear on the customer’s intent, consider including additional information to reduce the risk of problems later on.

The keyword “shall” A traditional convention is to use the keyword “shall” to describe some system capability. People sometimes object to the word “shall.” “That’s not how people talk,” they protest. So what? “Shall” statements clearly indicate the desired functionality, consistent with your overarching objective of clear and effective communication. You might prefer to say “must,” “needs to,” or something similar, but be consistent. I sometimes read specifications that contain a random and confusing mix of requirements verbs: shall, must, may, might, will, would, should, could, needs to, has to, should provide, and others. I never know if there are differences between the meanings of these or not. Nuances between different verbs also make the document far more difficult for cross-cultural teams to interpret consistently. You’re better off sticking with a keyword such as “shall.”

Some requirement authors deliberately use different verbs to imply subtle distinctions. They use certain keywords to connote priority: “shall” means required, “should” means desired, and “may” means optional (ISO/IEC/IEEE 2011). We regard such conventions as dangerous. It’s clearer to always say “shall” or “must” and explicitly assign high, medium, or low priority to each requirement. Also, priorities will change as iterations proceed, so don’t tie them to the phrasing of the requirements. Today’s “must” could become tomorrow’s “should.” Other authors use “shall” to indicate a requirement and “will” to denote a design expectation. Such conventions run the risk of some readers not understanding the distinctions between words people use interchangeably in everyday conversation; they are best avoided.

Trap One witty consultant suggested that you mentally replace each instance of “should” with “probably won’t.” Would the resulting requirement be acceptable? If not, replace “should” with something more precise.

Active voice Write in the active voice to make it clear what entity is taking the action described. Much business and scientific writing is in the passive voice, but it is never as clear and direct as using the active voice. The following requirement is written in passive voice:

Upon product upgrade shipment, the serial number will be updated on the contract line.

The phrasing “will be updated” is indicative of passive voice. It denotes the recipient of the action (serial number) but not the performer of the action. That is, this phrasing offers no clue as to who or what updates the serial number. Will the system do that automatically, or is the user expected to update the serial number? Rephrasing this requirement into active voice makes the actor explicit and also clarifies the triggering event:

When Fulfillment confirms that they shipped a product upgrade, the system shall update the customer's contract with the new product serial number.

Individual requirements Avoid writing long narrative paragraphs that contain multiple requirements. Readers shouldn't have to glean the individual requirements embedded in a mass of free-flowing descriptive language. Clearly distinguish individual requirements from background or contextual information. Such information is valuable to readers, but they need to unambiguously recognize the actual requirement statements. I once reviewed a large requirements specification written in the form of long paragraphs. I could read a full page and understand it, but I had to work hard to pick out the discrete requirements. Other readers might well come to different conclusions of exactly what requirements were lurking in that mass of text.

Words such as “and,” “or,” “additionally,” or “also” in a requirement suggest that several requirements might have been combined. This doesn't mean you can't use “and” in a requirement; just make sure the conjunction is joining two parts of a single requirement instead of two separate requirements. If you would use different tests to verify the two parts, split the sentence into separate requirements.

Avoid using “and/or” in a requirement; it leaves the interpretation up to the reader, as in this case:

The system must permit search by order number, invoice number, and/or customer purchase order number.

This requirement would permit the user to enter one, two, or three numbers at once when performing a single search. That might not be what's intended.

The words “unless,” “except,” and “but” also indicate the presence of multiple requirements:

The Buyer's credit card on file shall be charged for payment, unless the credit card has expired.

Failing to specify what happens when the “unless” clause is true is a common source of missing requirements. Split this into two requirements to address the behavior for the two conditions of the credit card being active and expired:

If the Buyer's credit card on file is active, the system shall charge the payment to that card.

and

If the Buyer's credit card on file has expired, the system shall allow the Buyer to either update the current credit card information or enter a new credit card for payment.

Level of detail

Requirements need to be specified at a level of precision that provides developers and testers with just enough information to properly implement them.

Appropriate detail An important part of requirements analysis is to decompose a high-level requirement into sufficient detail to clarify it and flesh it out. There's no single correct answer to the commonly asked question, "How detailed should the requirements be?" Provide enough specifics to minimize the risk of misunderstanding, based on the development team's knowledge and experience. The fewer the opportunities for ongoing discussion about requirements issues, the more specifics you need to record in the requirements set. If a developer can think of several possible ways to satisfy a requirement and all are acceptable, the specificity and detail are about right. You should include more detail when (Wiegers 2006):

- The work is being done for an external client.
- Development or testing will be outsourced.
- Project team members are geographically dispersed.
- System testing will be based on requirements.
- Accurate estimates are needed.
- Requirements traceability is needed.

It's safe to include less detail when:

- The work is being done internally for your company.
- Customers are extensively involved.
- Developers have considerable domain experience.
- Precedents are available, as when a previous application is being replaced.
- A package solution will be used.

Consistent granularity Requirement authors often struggle to find the right level of granularity for writing functional requirements. It's not necessary to specify all of your requirements to the same level of detail. For example, you might go into more depth in an area that presents higher risk than others. Within a set of related requirements, though, it's a good idea to try to write functional requirements at a consistent level of granularity.

A helpful guideline is to write individually testable requirements. The count of testable requirements has even been proposed as a metric for software product size (Wilson 1995). If you can think of a small number of related test cases to verify that a requirement was correctly implemented, it's probably at an appropriate granularity. If you envision numerous and diverse tests, perhaps several requirements are combined and ought to be separated.

I've seen requirement statements in the same SRS that varied widely in their scope. For instance, the following two functions were split out as separate requirements:

1. *The system shall interpret the keystroke combination Ctrl+S as File Save.*
2. *The system shall interpret the keystroke combination Ctrl+P as File Print.*

These requirements are very fine-grained. They will need few tests for verification of correct behavior. You can imagine a tediously long list of similar requirements, which would better be expressed in the form of a table that lists all the keystroke shortcuts and how the system interprets them.

However, that same SRS also contained a functional requirement that seemed rather large in scope:

The product shall respond to editing directives entered by voice.

This single requirement—seemingly no larger or smaller than all the others in the SRS—stipulated the inclusion of a complex speech-recognition subsystem—virtually an entire product in its own right! Verifying this one requirement in the working system could require hundreds of tests. The requirement as stated here could be appropriate at the high level of abstraction found in a vision statement or a market requirements document, but the speech-recognition requirement clearly demands much more functionality detail.

Representation techniques

Readers' eyes glaze over when confronting a dense mass of turgid text or a long list of similar-looking requirements. Consider the most effective way to communicate each requirement to the intended audience. Some alternatives to the natural language requirements that we're used to are lists, tables, visual analysis models, charts, mathematical formulas, photographs, sound clips, and video clips. These won't suffice as substitutes for written requirements in many cases, but they serve as excellent supplemental information to enhance the reader's understanding.



I once saw a set of requirements that fit the following pattern:

The Text Editor shall be able to parse <format> documents that define <jurisdiction> laws.

There were 3 possible values for <format> and 4 possible values for <jurisdiction>, for a total of 12 similar requirements. The SRS did indeed contain 12 such requirements, but one of the combinations was missing and another was duplicated. You can prevent such errors by representing these types of requirements in a table, which is more compact and less boring than a requirements list. The generic requirement could be stated as:

Editor.DocFormat *The Text Editor shall be able to parse documents in several formats that define laws in the jurisdictions shown in Table 11-1.*

TABLE 11-1 Requirements for parsing documents

<i>Jurisdiction</i>	<i>Tagged format</i>	<i>Untagged format</i>	<i>ASCII format</i>
<i>Federal</i>	.1	.2	.3
<i>State</i>	.4	.5	.6
<i>Territorial</i>	.7	N/A	.8
<i>International</i>	.9	.10	.11

The cells in the table contain only the suffix to append to the master requirement's identifier. For example, the third requirement in the top row expands to:

Editor.DocFormat.3 *The Text Editor shall be able to parse ASCII documents that define federal laws.*

If any of the combinations don't have a corresponding functional requirement for some logical reason, put *N/A* (not applicable) in that table cell. This is much clearer than omitting the irrelevant combination from the long list and then having a reader wonder why there is no requirement for parsing documents containing territorial laws in the untagged format. This technique also ensures completeness in the requirements set—if there's something in every cell, you know you haven't missed any.

Avoiding ambiguity

Requirements quality is in the eye of the reader, not the author. The analyst might believe that a requirement he has written is crystal clear, free from ambiguities and other problems. However, if a reader has questions, the requirement needs additional work. Peer reviews are the best way to find places where the requirements aren't clearly understood by all the intended audiences. This section describes several common sources of requirements ambiguity.

Fuzzy words Use terms consistently and as defined in the glossary. Watch out for synonyms and near-synonyms. I know of one project where four different terms were used to refer to the same item in a single requirements document. Pick a single term and use it consistently, placing synonyms in the glossary so people who are accustomed to calling the item by a different name see the connection.

If you use a pronoun to refer to something mentioned earlier, make sure the antecedent is crystal clear. Adverbs introduce subjectivity and hence ambiguity. Avoid words like *reasonably*, *appropriately*, *generally*, *approximately*, *usually*, *systematically*, and *quickly* because the reader won't be sure how to interpret them.

Ambiguous language leads to unverifiable requirements, so avoid using vague and subjective terms. Table 11-2 lists many such terms, along with suggestions for how to remove the ambiguity. Some of these words might be acceptable in business requirements, but not in user requirements or specific functional requirements that are attempting to describe the solution to be built.

TABLE 11-2 Some ambiguous terms to avoid in requirements

Ambiguous terms	Ways to improve them
acceptable, adequate	Define what constitutes acceptability and how the system can judge this.
and/or	Specify whether you mean “and,” “or,” or “any combination of” so the reader doesn’t have to guess.
as much as practicable	Don’t leave it up to the developers to determine what’s practicable. Make it a TBD and set a date to find out.
at least, at a minimum, not more than, not to exceed	Specify the minimum and maximum acceptable values.
best, greatest, most	State what level of achievement is desired and the minimum acceptable level of achievement.
between, from X to Y	Define whether the end points are included in the range.
depends on	Describe the nature of the dependency. Does another system provide input to this system, must other software be installed before your software can run, or does your system rely on another to perform some calculations or provide other services?
efficient	Define how efficiently the system uses resources, how quickly it performs specific operations, or how quickly users can perform certain tasks with the system.
fast, quick, rapid	Specify the minimum acceptable time in which the system performs some action.
flexible, versatile	Describe the ways in which the system must be able to adapt to changing operating conditions, platforms, or business needs.
i.e., e.g.	Many people are unclear about which of these means “that is” (i.e., meaning that the full list of items follows) and which means “for example” (e.g., meaning that just some examples follow). Use words in your native language, not confusing Latin abbreviations.
improved, better, faster, superior, higher quality	Quantify how much better or faster constitutes adequate improvement in a specific functional area or quality aspect.
including, including but not limited to, and so on, etc., such as, for instance	List all possible values or functions, not just examples, or refer the reader to the location of the full list. Otherwise, different readers might have different interpretations of what the whole set of items being referred to contains or where the list stops.
in most cases, generally, usually, almost always	Clarify when the stated conditions or scenarios do not apply and what happens then. Describe how either the user or the system can distinguish one case from the other.
match, equals, agree, the same	Define whether a text comparison is case sensitive and whether it means the phrase “contains,” “starts with,” or is “exact.” For real numbers, specify the degree of precision in the comparison.
maximize, minimize, optimize	State the maximum and minimum acceptable values of some parameter.
normally, ideally	Identify abnormal or non-ideal conditions and describe how the system should behave in those situations.
optionally	Clarify whether this means a developer choice, a system choice, or a user choice.
probably, ought to, should	Will it or won’t it?
reasonable, when necessary, where appropriate, if possible, as applicable	Explain how either the developer or the user can make this judgment.
robust	Define how the system is to handle exceptions and respond to unexpected operating conditions.

Ambiguous terms	Ways to improve them
seamless, transparent, graceful	What does “seamless” or “graceful” mean to the user? Translate the user’s expectations into specific observable product characteristics.
several, some, many, few, multiple, numerous	State how many, or provide the minimum and maximum bounds of a range.
shouldn’t, won’t	Try to state requirements as positives, describing what the system will do.
state-of-the-art	Define what this phrase means to the stakeholder.
sufficient	Specify how much of something constitutes sufficiency.
support, enable	Define exactly what functions the system will perform that constitute “supporting” some capability.
user-friendly, simple, easy	Describe system characteristics that will satisfy the customer’s usage needs and usability expectations.

The A/B construct Many requirements specifications include expressions in the form “A/B,” in which two related (or synonymous, or opposite) terms are combined with a slash. Such expressions frequently are ambiguous. Here’s an example:

The system shall provide automated information collection of license key data for a mass release from the Delivery/Fulfillment Team.

This sentence could be interpreted in several ways:

- The name of the team is Delivery/Fulfillment.
- Delivery and fulfillment are synonyms.
- Some projects call the group a Delivery Team; others call it a Fulfillment Team.
- Either the Delivery Team or the Fulfillment Team can do a mass release, so the slash means “or.”
- The Delivery Team and the Fulfillment Team jointly do a mass release, so the slash means “and.”

Sometimes authors use the A/B construct because they aren’t sure exactly what they have in mind. Unfortunately, this means that each reader gets to interpret the requirement to mean whatever he thinks it ought to mean. It’s better to decide exactly what you intend to say and choose the right words.

Boundary values Many ambiguities occur at the boundaries of numerical ranges in both requirements and business rules. Consider the following:

Vacation requests of up to 5 days do not require approval. Vacation requests of 5 to 10 days require supervisor approval. Vacation requests of 10 days or longer require management approval.

This phrasing makes it unclear as to which category vacation requests of exactly 5 days and exactly 10 days belong. It gets even more confusing if fractions are involved, like 5.5 days of vacation. The words “through,” “inclusive,” and “exclusive” make it totally clear whether the endpoints of the numerical range lie inside or outside the range:

Vacation requests of 5 or fewer days do not require approval. Vacation requests of longer than 5 days through 10 days require supervisor approval. Vacation requests of longer than 10 days require management approval.

Negative requirements People sometimes write requirements that say what the system *will not* do rather than what it *will* do. How do you implement a don't-do-this requirement? Double and triple negatives are particularly tricky to decipher. Try to rephrase negative requirements into a positive sense that clearly describes the restricting behavior. Here's an example:

Prevent the user from activating the contract if the contract is not in balance.

Consider rephrasing this double negative ("prevent" and "not in balance") as a positive statement:

The system shall allow the user to activate the contract only if the contract is in balance.

Instead of using negative requirements to indicate that certain functionality is out of scope, include the restriction in the Limitations and Exclusions section of the vision and scope document, as described in Chapter 5, "Establishing the business requirements." If a specific requirement was once in scope but then removed, you don't want to lose sight of it—it might come back someday. If you are maintaining requirements in a document, use strikethrough formatting to indicate a deleted requirement. The best way to handle such deleted requirements is with a requirements status attribute in a requirements management tool (see Chapter 27, "Requirements management practices," for more about requirements attributes and status tracking).

Avoiding incompleteness

We don't know of any way to be certain that you've found every requirement. Chapter 7, "Requirements elicitation," suggests several ways to identify missing requirements. Focusing elicitation on user tasks rather than system features can help avoid overlooking functionality. Also, using analysis models can help you spot missing requirements (see Chapter 12, "A picture is worth 1024 words").

Symmetry Symmetrical operations are a common source of missing requirements. I once found the following requirement in an SRS I was reviewing:

The user must be able to save the contract at any point during manual contract setup.

Nowhere in the rest of the specification did I find a requirement to allow the user to retrieve an incomplete but saved contract to work on it further: perhaps a requirement was missing. Nor was it clear whether the system should validate the data entries in the incomplete contract before saving it. An implied requirement? Developers need to know.

Complex logic Compound logical expressions often leave certain combinations of decision values undefined. Consider this requirement:

If the Premium plan is not selected and proof of insurance is not provided, the customer should automatically default into the Basic plan.

This requirement refers to two binary decisions, whose combinations lead to four possible outcomes. However, the specification only addressed this one combination. It didn't say what should happen if:

- The Premium plan is selected and proof of insurance is not provided.

- The Premium plan is selected and proof of insurance is provided.
- The Premium plan is not selected and proof of insurance is provided.

The reader is forced to conclude that the system doesn't take any action for those three other conditions. That might be correct, but it's better to make such conclusions explicit rather than implicit. Use decision tables or decision trees to represent complex logic and ensure that you have not missed any variants.

Missing exceptions Each requirement that states how the system should work when everything is correct should also have accompanying requirements as necessary to describe how the system should respond when exceptions occur. Consider the following requirement:

If the user is working in an existing file and chooses to save the file, the system shall save it with the same name.

This requirement alone does not indicate what the system should do if it's unable to save the file with the same name. An appropriate second requirement to go with the first might be:

If the system is unable to save a file using a specific name, the system shall give the user the option to save it with a different name or to cancel the save operation.

Sample requirements, before and after

This chapter opened with several characteristics of high-quality requirements. Requirements that don't exhibit these characteristics cause confusion, wasted effort, and rework later, so strive to correct any problems early. Following are several functional requirements adapted from real projects that are less than ideal. Examine each statement for those quality characteristics to see whether you can spot the problems. Verifiability is a good starting point. If you can't devise tests to tell whether the requirement was correctly implemented, it's probably ambiguous or lacks necessary information.

For each example, we present some observations about the problems with these requirements and suggested improvements. Additional reviews would no doubt improve them further, but at some point you need to write software. More examples of rewriting poor requirements are available from Ivy Hooks and Kristin Farry (2001), Al Florence (2002), Ian Alexander and Richard Stevens (2002), and Karl Wiegers (2006). Note that pulling requirements out of context like this shows them at their worst. These might well make more sense in their original environment. We also assume that business analysts (and all other team members) come to work each day to do the best job they can, based on what they know at the moment, so we're not picking on the original authors here.

Trap Watch out for analysis paralysis. All of the sample "after" requirements in this chapter can be improved further, but you can't spend forever trying to perfect the requirements. Remember, your goal is to write requirements that are *good enough* to let your team proceed with design and construction at an acceptable level of risk.

Example 1 *The Background Task Manager shall provide status messages at regular intervals not less than every 60 seconds.*

What are the status messages? Under what conditions and in what fashion are they provided to the user? If displayed on the screen, how long do they remain visible? Is it okay if they just flash up for half a second? The timing interval is not clear, and the word “every” just muddles the issue. One way to evaluate a requirement is to see whether a ludicrous but legitimate interpretation is all right with the user. If not, the requirement needs more work. In this example, is the interval between status messages supposed to be *at least* 60 seconds, so providing a new message once per year is okay? Alternatively, if the intent is to have *at most* 60 seconds elapse between messages, would one millisecond be too short an interval? These extreme interpretations might be consistent with the original requirement, but they certainly aren’t what the user had in mind. Because of these problems, this requirement is not verifiable.

Here’s one way to rewrite the preceding requirement to address those shortcomings, after we get some more information from the customer:

1. The Background Task Manager (BTM) shall display status messages in a designated area of the user interface.

1.1. The BTM shall update the messages every 60 plus or minus 5 seconds after background task processing begins.

1.2. The messages shall remain visible continuously during background processing.

1.3. The BTM shall display the percent of the background task that is completed.

1.4. The BTM shall display a “Done” message when the background task is completed.

1.5. The BTM shall display a message if the background task has stalled.

Rewriting a flawed requirement often makes it longer because information was missing. Splitting this into multiple child requirements makes sense because each will demand separate tests. This also makes each one individually traceable. There would likely be additional status messages that the BTM might display. If those are documented someplace else, such as in an interface specification, incorporate that information here by reference instead of replicating it. Listing the messages in a table of conditions and corresponding messages would be more concise than writing numerous functional requirements.

The revised requirements don’t specify how the status messages will be displayed, just “in a designated area of the user interface.” Such wording defers the placement of the messages to being a design issue, which is fine in many cases. If you specify the display location in the requirements, it becomes a design constraint placed on the developer. Unnecessarily constrained design options frustrate the programmers and can result in a suboptimal product design.

Suppose, though, that we’re adding this functionality to an existing application whose user interface already contains a status bar, where users are accustomed to seeing important messages. For consistency with the rest of the application it would make perfect sense to stipulate that the BTM’s status messages shall appear in the status bar. That is, you might deliberately impose the design constraint for a very good reason.

Example 2 *Corporate project charge numbers should be validated online against the master corporate charge number list, if possible.*

The phrase “if possible” is ambiguous. Does it mean “if it’s technically feasible” (a question for the developer) or “if the master charge number list can be accessed at run time”? If you aren’t sure whether a requested capability can be delivered, use TBD to indicate that the issue is unresolved. After investigation, either the TBD goes away or the requirement goes away. This requirement doesn’t specify what to do when the validation either passes or fails. Also, avoid imprecise words such as “should.” Here’s a revised version of this requirement:

At the time the requester enters a charge number, the system shall display an error message if the charge number is not in the master corporate charge number list.

A related requirement would address the exception condition of the master corporate charge number list not being available at the time the validation was attempted.

Example 3 *The device tester shall allow the user to easily connect additional components, including a pulse generator, a voltmeter, a capacitance meter, and custom probe cards.*

This requirement is for a product containing embedded software that’s used to test several kinds of measurement devices. The word “easily” implies a usability requirement, but it is neither measurable nor verifiable. “Including” doesn’t make it clear whether this is the complete list of external devices that must be connected to the tester. Perhaps there are many others that we don’t know about. Consider the following alternative requirements, which contain some intentional design constraints:

- 1. The device tester shall incorporate a USB port to allow the user to connect any measurement device that has a USB connection.*
- 2. The USB port shall be installed on the front panel to permit a trained operator to connect a measurement device in 10 seconds or less.*

A business analyst shouldn’t rewrite requirements in a way that imposes design constraints on his own initiative. Instead, detect the flawed requirements and discuss them with the appropriate stakeholders so they can be clarified.

Example 4 *The system must check for inconsistencies in account data between the Active Account Log and the Account Manager archive. The logic that is used to generate these comparisons should be based on the logic in the existing consistency checker tool. In other words, the new code does not need to be developed from scratch. The developers should utilize the current consistency checker code as the foundation. However, additional logic must be added to identify which database is the authoritative source. The new functionality will include writing data to holding tables to indicate how/where to resolve inconsistencies. Additionally, the code should also check for exception scenarios against the security tools database. Automated email alerts should be sent to the Security Compliance Team whenever discrepancies are found.*

This is a good one for you to practice on. We’ll point out some of the problems with this paragraph, and you might want to try rewriting it in an improved form, making some assumptions as necessary to fill in the gaps. Following are some issues you might want to correct.

- There are numerous requirements in here that should be split out individually.
- If the comparison logic is “based on” logic in the existing consistency checker tool, exactly what portion of the code can be reused and how does it need to be changed? What functions are different between the new system and the existing tool? What “additional logic” must be added? How exactly can the system determine “which database is the authoritative source”?
- The new functionality “includes” writing data to holding tables; is that all, or is other functionality “included” that isn’t explicitly stated?
- Clarify what “how/where” means when resolving inconsistencies.
- “Should” is used in several places.
- What’s the relationship between an “exception scenario” and a “discrepancy”? If they’re synonyms, pick one term and stick with it. A glossary might clarify whether these are the same or how they are related.
- What information should the system send to the Security Compliance Team when it detects a discrepancy?

As we said earlier, you’re never going to get perfect requirements. But an experienced BA can nearly always help make requirements better.



Next steps

- Hold a discussion with your customers, developers, and testers to evaluate the current level of requirements documentation on your project to determine if more or less detail is needed in specific areas and how best to represent those requirements.
- Examine a page of functional requirements from your project’s requirements set to see whether each statement exhibits the characteristics of excellent requirements. Look for any of the types of problems described in this chapter. Rewrite any requirements that don’t measure up.
- Convene three to six project stakeholders to inspect the SRS for your project (Wieggers 2002). Make sure each requirement demonstrates the desirable characteristics discussed in this chapter. Look for conflicts between different requirements in the specification, for missing requirements, and for missing sections of the SRS. Ensure that the defects you find are corrected in the SRS and in any downstream work products based on those requirements.